HEALTH CARE LAB TESTING BOOKING & REPORT MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

AMANJEET(23BCS10270)
AMBUJ(23BCS10884)
BHUVANESH(23BCS11269)
KUMAR SHANU(23BCS11852)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPTER SCIENCE ENGINEERING

Chandigarh University

April 2026

BONAFIDE CERTIFICATE

Certified that this project report **"HEALTH CARE LAB TESTING BOOKING & REPORT MANAGEMENT SYSTEM"** is the Bonafide work of **"AMANJEET(23BCS10270), AMBUJ(23BCS10884), BHUVANESH(23BCS11269),KUMAR SHANU(23BCS11852))**who carried out the project work under my/our supervision.

SIGNATURE OF HOD

Dr.Sanjiv Kumar

HEAD OF THE DEPARTMENT

BE-CSE

SIGNATURE OF THE SUPERVISOR

Er. Rahul Durgapal

SUPERVISOR

Assistant Professor

BE-CSE

Submitted for the project viva-voce examination held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

Acknowledgement

We express our sincere gratitude to **Chandigarh University** for providing us with the opportunity to undertake this project, **"Health Care Lab Testing Booking & Report Management System,"** as a part of our academic curriculum. This project has been a valuable learning experience, allowing us to apply theoretical concepts of software development, database management, web technologies, and system design to a real-world healthcare problem.

We would like to convey our heartfelt thanks to our respected **Head of Department, Dr.Sanjiv Kumar**, and our project **Supervisor, Er. Rahul Durgapal, Assistant Professor, BE-CSE**, for their constant guidance, encouragement, and support throughout the development of this project. Their valuable suggestions and academic supervision helped us at every stage, from planning and design to implementation and documentation.

We are also thankful to the faculty members of the **Department of Computer Science Engineering** for providing us with the necessary academic environment, technical knowledge, and motivation required for the successful completion of this work.

This project helped us gain practical exposure in **Spring Boot, React, TypeScript, MySQL, JWT security, role-based access control, PDF report generation, and AI-based report analysis**, while also improving our teamwork, problem-solving, and project management skills. Working on this system deepened our understanding of how technology can improve efficiency, transparency, and service quality in the healthcare sector.

We would also like to thank our friends and family for their continuous encouragement, moral support, and motivation during the completion of this project.

Finally, we express our gratitude to everyone who directly or indirectly contributed to the successful completion of this project report.

Thank you.

Sincerely,
AMANJEET (23BCS10270)
AMBUJ (23BCS10884)
BHUVANESH (23BCS11269)
KUMAR SHANU (23BCS11852)

TABLE OF CONTENTS

List of Tables

List of Figure

ABSTRACT

The Health Care Lab Testing Booking & Report Management System is a full-stack web application designed to streamline the end-to-end workflow of diagnostic laboratory operations in the Indian healthcare market. The system addresses the fragmented and manual processes that currently exist in pathology labs, diagnostic centers, and hospitals by providing a unified digital platform that serves multiple stakeholder roles: patients, laboratory technicians, medical officers, and system administrators.

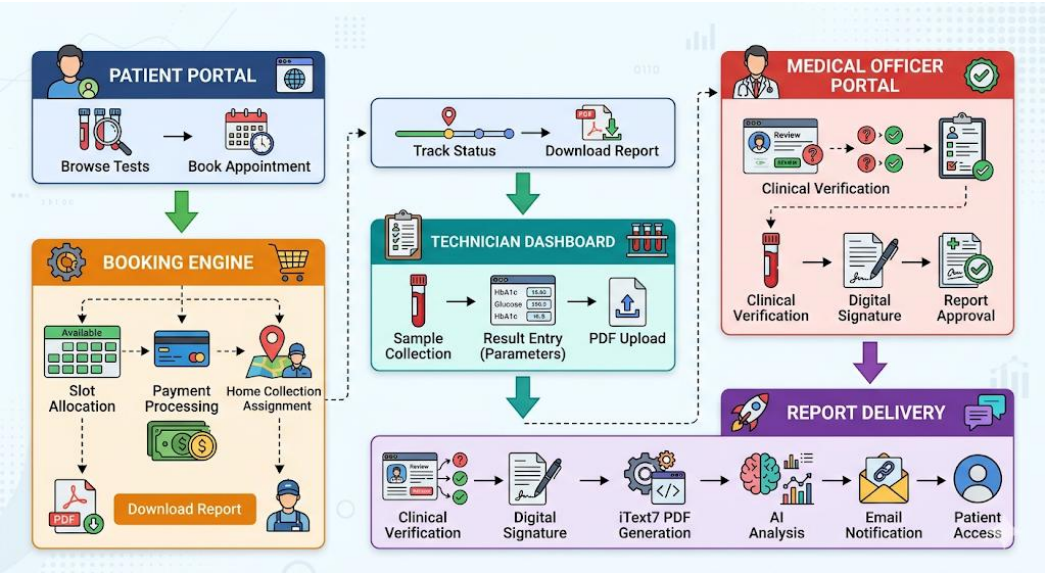
The application enables patients to browse a comprehensive catalog of over 1,000 lab tests and health packages, book appointments with home sample collection support, track their booking status in real time, and download verified diagnostic reports. Technicians are provided a mobile-friendly dashboard to manage sample collection, enter test result values for individual parameters, and upload finalized reports. Medical officers perform clinical verification of submitted results, provide digital signatures with remarks, and authorize report delivery to patients. The administrative interface provides complete platform oversight including staff management, booking management, revenue analytics, audit trails, and promo code administration.

The backend is implemented using Java Spring Boot with a layered architecture comprising 37 REST controllers, 55 business logic services, and 37 JPA repositories connected to a MySQL relational database with over 50 normalized tables. Flyway manages versioned database migrations ensuring schema consistency across deployments. The frontend is built using React 18, TypeScript, Vite, and Tailwind CSS, delivering a responsive progressive web application with 35+ pages and 127 components organized around role-specific workflows. Security is enforced through JSON Web Tokens (JWT) and role-based access control (RBAC) protecting all sensitive API endpoints.

Key advanced features include AI-assisted report pattern recognition using the Anthropic Claude API, automated PDF Smart Report generation using iText7, reflex testing rule evaluation, digital consent capture for sensitive tests, real-time notification delivery, and a comprehensive audit logging system aligned with healthcare compliance standards. The system architecture is containerized using Docker and is designed for deployment on cloud infrastructure with horizontal scalability.

GRAPHICAL ABSTRACT

The graphical abstract below represents the complete end-to-end operational flow of the Health Care Lab Testing Booking & Report Management System:



The system operates on a Spring Boot REST API backend (port 8080) connected to a MySQL database, serving a React/TypeScript frontend (port 3000) over HTTPS. All communications are secured with JWT authentication and RBAC enforcement at both the API gateway level and method level using Spring Security @PreAuthorize annotations.

Abbreviations

- **AI**: Artificial Intelligence
- **API**: Application Programming Interface
- **JWT**: JSON Web Token
- **RBAC**: Role-Based Access Control
- **PDF**: Portable Document Format
- **SQL**: Structured Query Language
- **MO**: Medical Officer
- **UI**: User Interface
- **REST**: Representational State Transfer
- **JPA**: Java Persistence API
- **ORM**: Object Relational Mapping
- **CORS**: Cross-Origin Resource Sharing

Keywords

Health Care Lab Testing, Lab Booking System, Report Management, Spring Boot, React, MySQL, JWT Authentication, Role-Based Access Control, Smart Report, AI Health Insights, PDF Report Generation, Diagnostic Workflow Automation.

CHAPTER - 1

INTRODUCTION

1.1 Project Background and Problem Statement

The Indian diagnostic and pathology laboratory market is one of the fastest-growing segments of the healthcare industry, valued at approximately USD 9 billion and projected to reach USD 32 billion by 2030. With over 100,000 registered diagnostic laboratories across the country, the sector processes millions of test orders daily. Despite this scale, a significant proportion of the industry continues to rely on paper-based or rudimentary software systems for managing patient bookings, sample collection logistics, result recording, and report generation.

The current state of lab management presents several critical challenges:

- Fragmented Booking: Patients book tests through phone calls, walk-ins, or third-party aggregators with no unified tracking or status visibility.
- Manual Result Entry: Technicians record test values in paper worksheets or disconnected spreadsheets, leading to transcription errors and data loss.
- Delayed Report Delivery: Reports are physically printed or emailed as unsecured PDF attachments with no verification trail.
- No Clinical Oversight Integration: Medical officers verify reports through informal communication channels with no standardized workflow.
- Zero Audit Trails: There is no systematic record of who performed which action at what time, creating compliance and accountability gaps.
- Siloed Systems: Patient data, lab data, billing data, and communication are managed in separate disconnected tools.

The Health Care Lab Testing Booking & Report Management System addresses all of these problems through a unified, role-aware web platform that digitizes the complete diagnostic workflow from initial patient booking to final verified report delivery.

1.2 Identification of Problem

The primary objectives of this project are:

1. Design and implement a multi-role web application supporting Patient, Technician, Medical Officer, and Administrator workflows within a single integrated platform.
2. Develop a comprehensive lab test and health package catalog with advanced search, filtering, and category-based browsing capabilities.
3. Implement an end-to-end booking management system with home sample collection support, slot allocation, payment tracking, and real-time status updates.

4. Create a technician-facing mobile-friendly dashboard for sample collection tracking, test result parameter entry, and report document upload.
5. Build a medical officer verification portal with clinical notes, digital signature support, delta check (historical comparison), and report amendment workflow.
6. Develop an administrative command center with user lifecycle management, staff creation, booking oversight, revenue analytics, promo code management, and comprehensive audit logging.
7. Implement automated PDF Smart Report generation using iText7 with patient details, parameter values, reference ranges, and medical officer digital signature.
8. Integrate AI-assisted health insights using the Anthropic Claude API for plain-language clinical pattern recognition from verified test results.
9. Enforce enterprise-grade security through JWT authentication, BCrypt password hashing, RBAC authorization, rate limiting, and CORS configuration.
10. Ensure production readiness through Docker containerization, environment variable management, and comprehensive audit trails.

1.3 Identification of Tasks

The scope of the Health Care Lab Testing Booking & Report Management System encompasses the following functional domains:

- Patient Management: Registration, login, profile management, family member management, address book, booking history, and report access.
- Test Catalog Management: 1,000+ individual lab tests and 150+ health packages organized by category, with pricing, discount, fasting requirements, and parameter information.
- Booking and Scheduling: Real-time slot availability, home collection booking, sample collection address management, and booking status lifecycle management.
- Laboratory Operations: Technician assignment, sample collection tracking, test result entry by parameter, report file upload, and specimen rejection workflow.
- Clinical Verification: Medical officer review queue, result verification with remarks, digital signature, report amendment, critical value flagging, and patient notification.
- Report Management: PDF generation, AI analysis, Smart Report viewer, report download, and shared report access.
- Administration: User management, staff creation/removal, booking oversight, revenue charts, promo code CRUD, audit log viewer, and notification management.

1.4 Project Timeline

Phase	Activities	Deliverables	Duration
Phase 1	Requirements analysis, technology selection, database schema design	ER diagram, API contract, tech stack decision	2 weeks
Phase 2	Backend: Spring Boot setup, Auth module, User/Booking/Test APIs	Working REST API with JWT auth	3 weeks
Phase 3	Frontend: React setup, Patient portal, Test listing, Cart, Booking flow	Patient-facing pages fully functional	3 weeks
Phase 4	Technician dashboard, MO verification portal, Admin panel	All role dashboards complete	3 weeks
Phase 5	PDF report generation, AI analysis, notifications, audit logs	End-to-end report workflow complete	2 weeks
Phase 6	Security hardening, testing, bug fixes, Docker containerization	Production-ready deployment	2 weeks

Table 1: Project Timeline Overview

1.5 Organization of the Report

The remainder of this report is organized as follows:

- Chapter 2 presents a literature review of existing healthcare booking and laboratory information systems, analyzing their strengths, limitations, and the gaps that motivated this project.
- Chapter 3 describes the complete system design including architecture diagrams, technology stack decisions, database schema design, role-based access control model, and API design specifications.
- Chapter 4 covers the implementation details of both the backend and frontend systems, including key code structures, database implementation, and screenshots of the completed application.
- Chapter 5 presents the conclusion summarizing achievements and a discussion of future enhancement opportunities.

CHAPTER - 2

LITERATURE REVIEW / BACKGROUND STUDY

2.1 Timeline of the Reported Problem

Earlier, diagnostic lab processes were mostly manual, causing delays, poor coordination, and low transparency. With digital healthcare growth and increased demand for home sample collection, online lab booking systems became popular. However, most platforms still focus only on booking and report delivery, not the complete workflow. Small and medium laboratories still lack an affordable system that manages the full process in one platform.

2.2 Existing Healthcare Booking Systems

Several commercial and open-source healthcare laboratory management systems exist in the market. A systematic review of these platforms provides context for the design decisions made in this project.

Labs Platform

TATA 1mg operates India's largest online diagnostic booking platform, serving over 40 lakh customers. The platform offers an extensive test catalog with home sample collection and a 'Smart Report' feature that provides AI-assisted health insights alongside raw test data. Key strengths include a well-designed patient experience and comprehensive test coverage. Limitations include lack of a transparent technician workflow interface and no end-to-end clinical verification trail accessible to patients.

Practo Diagnostics

Practo integrates diagnostic booking with doctor consultation, creating a unified healthcare marketplace. The platform excels at doctor-test referral workflows but focuses primarily on appointment scheduling rather than the full sample collection and report generation lifecycle. It does not expose a workflow interface for laboratory technicians or medical officers.

Thyrocare LIS (Laboratory Information System)

Thyrocare's proprietary LIS handles high-volume test processing across a hub-and-spoke collection model. The system is highly optimized for bulk processing but is not publicly accessible and does not serve direct patient-facing digital workflows. Integration with external booking channels is handled through B2B APIs rather than a unified patient portal.

Open-Source LIMS (e.g., Bahmni, OpenELIS)

Open-source laboratory information management systems such as Bahmni (a distribution of OpenMRS for complex environments) and OpenELIS provide robust clinical data management capabilities. These systems are primarily designed for hospital environments in low-resource settings and require significant customization for direct-to-consumer lab booking. They lack modern web application design standards and role-specific mobile-friendly dashboards.

2.3 Bibliometric Analysis

Recent studies show growing interest in digital healthcare systems, laboratory information systems, AI-assisted diagnostics, and workflow automation. Most research focuses on booking, records, or reporting, but fewer systems combine patient booking, technician tasks, medical officer verification, and AI-based report interpretation in one role-based platform.

2.4 Review Summary

The review shows that existing systems provide useful features, but no single publicly accessible platform combines complete diagnostic workflow management, role-specific dashboards, AI insights, and affordability for smaller laboratories.

2.4.1 Comparative Analysis

Feature	TATA 1mg	Practo	Thyrocare	OpenELIS	This Project
Patient Booking Portal	Yes	Yes	Partial	No	Yes
Home Collection	Yes	No	Yes	No	Yes
Technician Dashboard	No	No	Internal	Partial	Yes
MO Verification Portal	No	No	No	Yes	Yes
Result Parameter Entry	No	No	Internal	Yes	Yes
AI Health Insights	Yes	No	No	No	Yes
PDF Smart Report	Yes	No	No	Yes	Yes
Admin Analytics	Internal	Yes	Internal	Partial	Yes
Open Source / Accessible	No	No	No	Yes	Yes
Audit Log / Compliance	Partial	No	No	Yes	Yes

Table 2: Comparative Analysis of Existing Systems

2.5 Problem Definition

Based on the review of existing systems and identified market gaps, the following problem definition was established for this project:

No existing publicly accessible system integrates the complete diagnostic lab workflow - from patient self-service booking through technician sample collection and result entry to medical officer clinical verification and AI-enhanced report delivery - into a single unified, role-aware web application accessible to small and medium diagnostic laboratories without enterprise licensing costs.

2.6 Goals and Objectives

1. Deliver a production-ready full-stack web application covering all four operational roles: Patient, Technician, Medical Officer, and Administrator.
2. Implement structured booking status lifecycle: BOOKED > SAMPLE_COLLECTED > PROCESSING > PENDING_VERIFICATION > VERIFIED > COMPLETED.
3. Enable parameterized test result entry with reference range comparison and automatic abnormal flagging.
4. Provide AI-powered health insights via integration with Anthropic Claude API on verified reports.
5. Achieve enterprise-grade security with JWT, RBAC, BCrypt, rate limiting, and comprehensive audit trails.
6. Package for Docker deployment with environment variable management for production readiness.

CHAPTER - 3

DESIGN FLOW/PROCESS

3.1. Evaluation & Selection of Specifications/Features

The system employs a three-tier client-server architecture: a React/TypeScript Single Page Application (frontend), a Java Spring Boot REST API (backend), and a MySQL relational database. The architecture is designed for scalability, separation of concerns, and independent deployment of each tier.

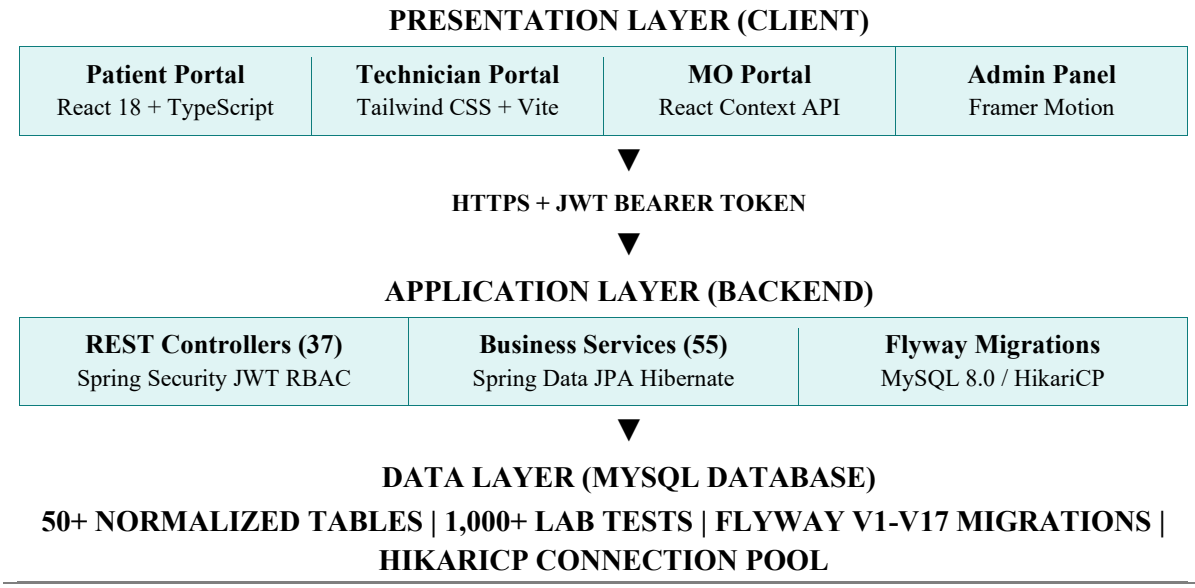


Figure 1: System Architecture Diagram

3.2. Design Constraints

Category	Technology	Purpose
Frontend Framework	React 18 + TypeScript	Component-based UI with type safety
Build Tool	Vite 5	Fast HMR and optimized production builds
CSS Framework	Tailwind CSS v4	Utility-first responsive styling
Animation	Framer Motion	Page transitions and UI animations
HTTP Client	Axios	API requests with interceptors and token management
State Management	React Context API + useState	Auth, Cart, Modal, Notification contexts
Backend Framework	Spring Boot 3.x (Java 17)	REST API with auto-configuration
Security	Spring Security + JWT	Stateless authentication with RBAC enforcement

ORM	Spring Data JPA / Hibernate	Database abstraction with entity mapping
DB Migration	Flyway	Versioned schema migrations (V1-V17)
Database	MySQL 8.0	Relational RDBMS with 50+ tables
PDF Generation	iText7	Smart Report PDF with charts and signatures
AI Integration	Groq	AI-assisted health insights from test results
Email	JavaMail/ Gmail's SMTP	Transactional booking and report notifications
Containerization	Docker + docker-compose	Production deployment packaging

Table 3: Technology Stack Summary

3.3 Analysis of Features and Finalization Subject to Constraints

The database follows Third Normal Form (3NF) with proper referential integrity constraints and indexed foreign keys. The schema is managed through Flyway versioned migrations ensuring reproducible deployments.

Table Name	Key Columns	Purpose
users	id, name, email, role, is_active	Patient, Technician, MO, Admin accounts
tests (lab_tests)	id, test_name, slug, category, price, is_active	Individual diagnostic test catalog (1,000+)
test_packages	id, package_name, package_code, total_price	Health checkup bundles (150+ packages)
bookings	id, patient_id, test_id, status, booking_date	Central booking record with status lifecycle
report_results	id, booking_id, parameter_id, result_value	Individual test parameter result values
report_verifications	id, booking_id, mo_id, status, clinical_notes	MO verification records with digital signature
reports	id, booking_id, report_pdf, status	Generated PDF report storage
test_parameters	id, test_id, parameter_name, unit, normal_range	Parameters for each test type
cart / cart_items	id, user_id, test_id, quantity	Shopping cart for booking workflow
audit_logs	id, user_id, action, entity_type, timestamp	Comprehensive action audit trail

Table 4: Key Database Tables and Purpose

3.4 Design Flow

The system implements four distinct roles, each with a defined set of permissions and a dedicated user interface:

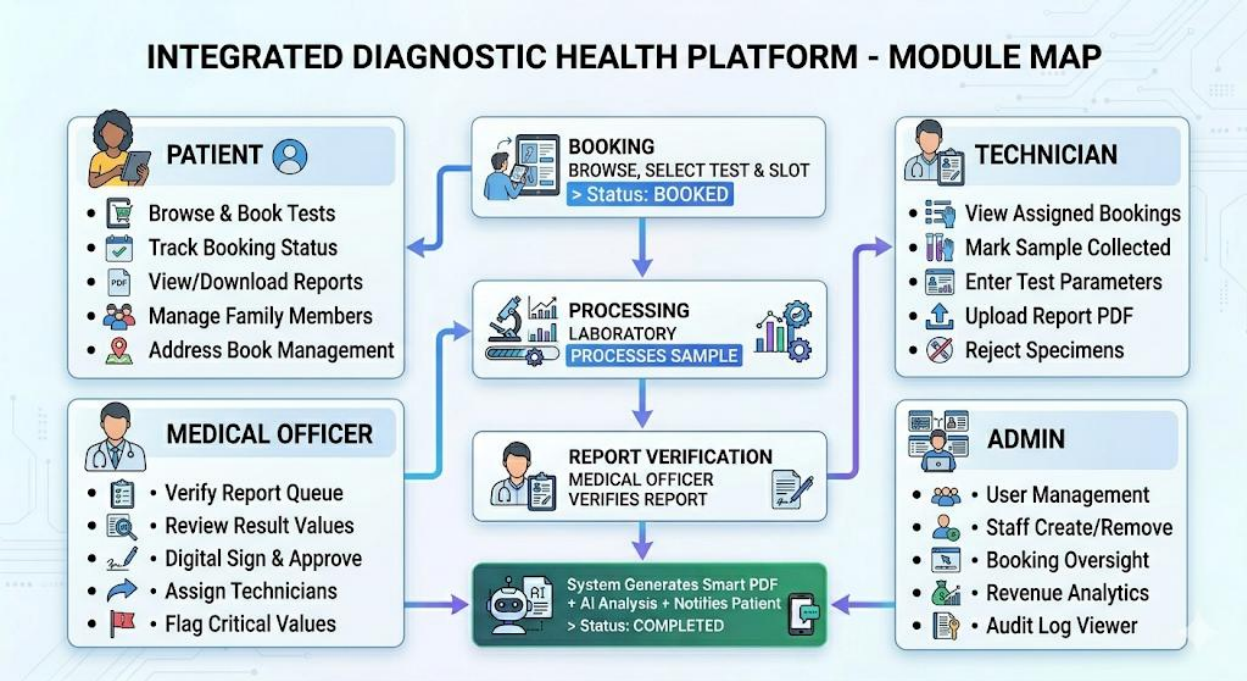


Figure 2: Booking Process Flowchart

3.5 Design selection

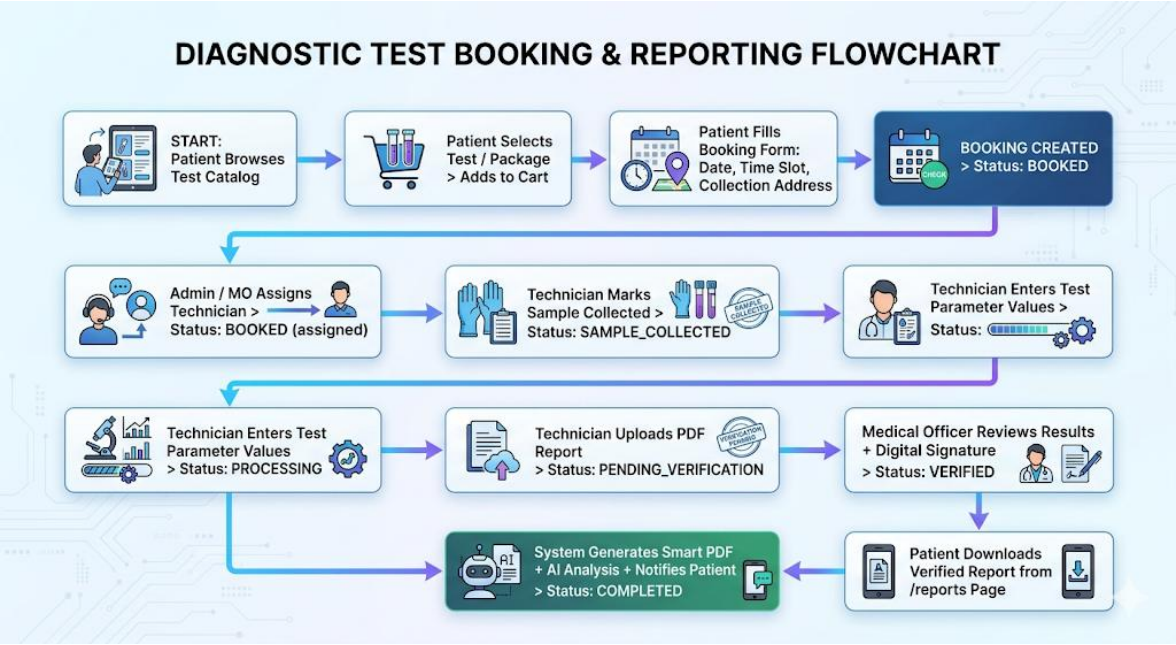


Figure 3: Booking Process Flowchart

3.6 Implementation plan/methodology

The project was implemented in phases. First, system requirements were analyzed for Patient, Technician, Medical Officer, and Administrator roles. Then, the system architecture was designed using the MERN stack.

After that, the database, APIs, and authentication system were developed. Separate dashboards were created for each role. The booking workflow, result entry, verification process, and AI health insights were then implemented step by step.
Finally, the system was tested, secured, and prepared for deployment using Docker and environment variables.

The REST API follows standard HTTP conventions with JSON request/response bodies. All responses use a consistent ApiResponse wrapper:

Method	Endpoint	Role	Description
POST	/api/auth/register	PUBLIC	Patient self-registration
POST	/api/auth/login	PUBLIC	JWT token generation
GET	/api/bookings/my	PATIENT	Get authenticated user bookings
PUT	/api/bookings/{id}/cancel	PATIENT	Cancel booking by ID
PUT	/api/bookings/{id}/collection	TECHNICIAN	Mark sample collected
POST	/api/reports/results	TECHNICIAN	Submit test parameter values
POST	/api/reports/upload	TECHNICIAN	Upload PDF report file
POST	/api/mo/verify/{bookingId}	MEDICAL_OFFICER	Verify and sign report
GET	/api/mo/pending	MEDICAL_OFFICER	Get pending verification queue
GET	/api/admin/stats	ADMIN	Platform statistics summary
POST	/api/admin/staff	ADMIN	Create technician/MO account
GET	/api/reports/{id}/download	PATIENT+	Download verified report PDF

Table 5: API Endpoint Overview (Selected)

CHAPTER - 4

IMPLEMENTATION AND RESULTS

4.1 Implementation of solution

The backend is implemented as a Maven-based Spring Boot 3.x application running on Java 17. The application follows a strict four-layer architecture:

- Controller Layer (37 controllers): Handles HTTP routing, request validation via @Valid, role enforcement via @PreAuthorize, and delegates to the service layer. All controllers are annotated with @RestController and @RequestMapping.
- Service Layer (55 services): Encapsulates business logic, transaction management (@Transactional), and inter-service communication. Key services include BookingService, MedicalOfficerService, PdfReportService, ReportGeneratorService, and AiAnalysisService.
- Repository Layer (37 repositories): Spring Data JPA interfaces extending JpaRepository with custom query methods derived from naming conventions and @Query annotations for complex queries.
- Entity Layer (45+ entities): JPA entities mapped to MySQL tables using @Entity, @Table, @Column, @ManyToOne, @OneToMany, and @ManyToMany annotations with Lombok @Data, @Builder for cleaner code.

Security Implementation

Security is implemented using Spring Security 6 with stateless JWT authentication:

- JwtAuthenticationFilter: Intercepts every request, validates the Bearer token, extracts the user email and role, and populates the SecurityContext.
- @PreAuthorize annotations: Applied to all controller methods requiring specific roles (e.g., @PreAuthorize("hasRole('ADMIN')")).
- BCryptPasswordEncoder: All user passwords are encoded using BCrypt with strength factor 10, preventing rainbow table attacks.
- RateLimitingFilter: Prevents brute-force attacks by enforcing per-IP request rate limits.
- CORS Configuration: Explicitly allows requests from frontend origins (localhost:3000, localhost:5173) in development and configurable production origins.

Booking Status Lifecycle

The booking status transitions are enforced by a validation map in BookingService that prevents invalid state changes:

- BOOKED > SAMPLE_COLLECTED or CANCELLED
- SAMPLE_COLLECTED > PROCESSING
- PROCESSING > PENDING_VERIFICATION
- PENDING_VERIFICATION > VERIFIED or PROCESSING (returned for correction)
- VERIFIED > COMPLETED
- Any state > CANCELLED (ADMIN override only)

4.1.1 Frontend Implementation (React)

The frontend is a Vite-powered React 18 TypeScript Single Page Application with code-splitting via React.lazy() and route-based lazy loading for all 35+ pages. The application serves four distinct user experiences based on the authenticated user role.

Patient Interface

The patient portal provides a consumer-grade experience comparable to commercial health platforms. Key features include a landing page with promotional banners and category navigation, a test listing page with advanced search and filter capabilities (category, price range, popularity), individual test detail pages with parameter information, package listing with Silver/Gold/Platinum tier filtering, a cart and checkout flow with promo code support, and a booking status tracker with a visual horizontal node pipeline showing each stage of the booking lifecycle.

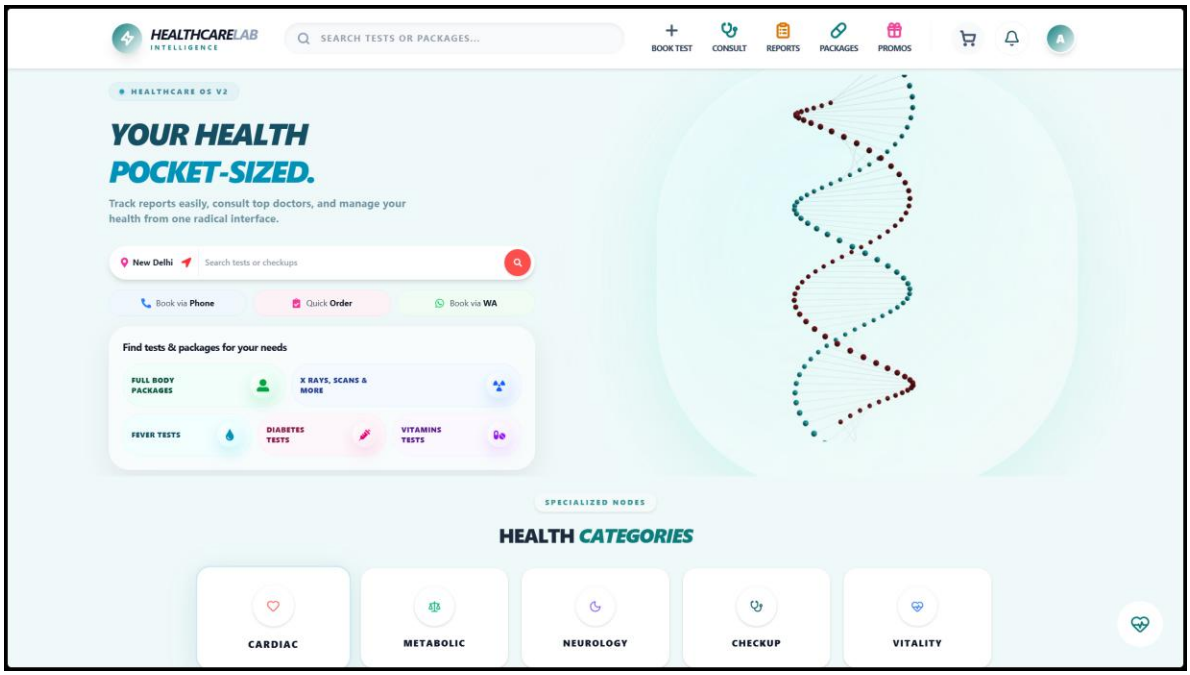


Figure 4 Patient Page Screenshot

Technician Interface

The technician dashboard is designed for mobile-friendly use in field operations. It presents assigned bookings organized by status with action buttons appropriate to each stage. The core workflow is: Mark Collected > Start Processing > Enter Results (dedicated result entry page with all test parameters) > Upload Report PDF. A visual node pipeline on each booking card shows the current stage with color-coded progression (green for complete nodes, orange for active, grey for upcoming, red for overdue).

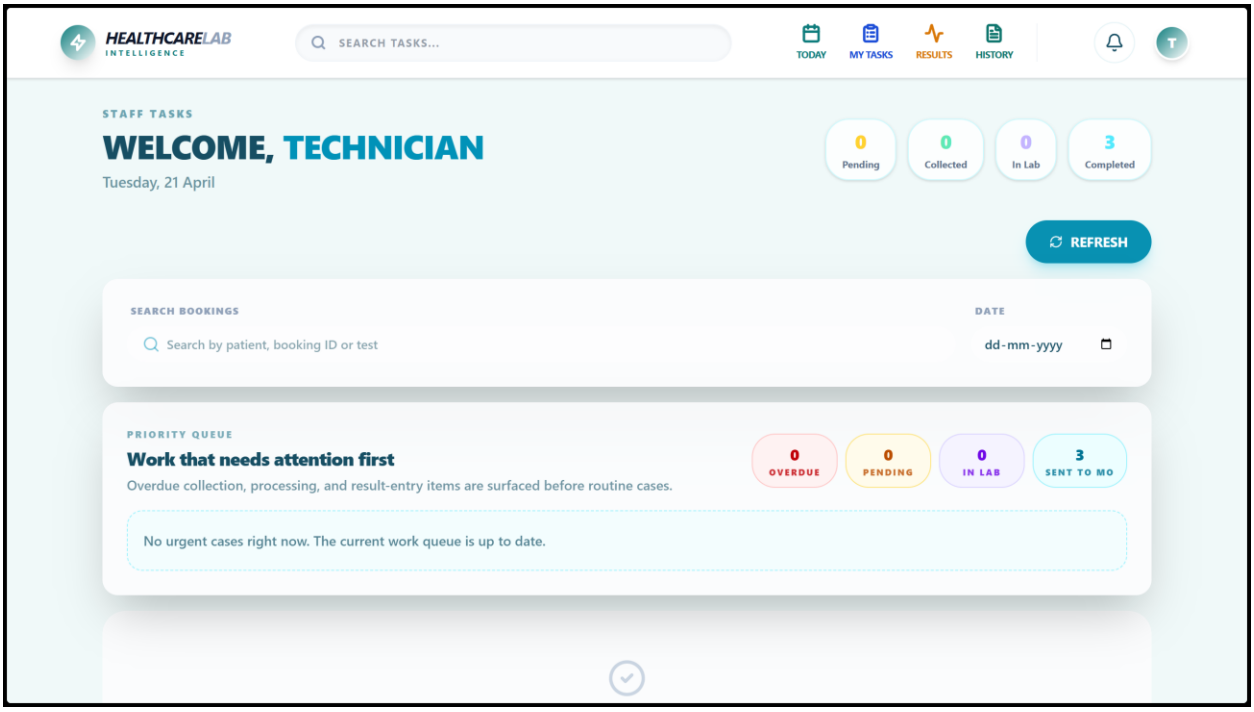


Figure 5 Technician Dashboard Screenshot

Medical Officer Interface

The medical officer portal provides a verification queue of pending reports. Each queued item can be expanded to reveal the full test result table with parameter values, reference ranges, and automatic HIGH/LOW/CRITICAL flags. The verification form requires clinical notes (minimum 10 characters), generates an automatic digital signature, and sends the approval to the backend which triggers PDF generation, AI analysis, and patient notification. The interface also includes an assignments page for technician allocation and a pipeline view for monitoring all active bookings by stage.

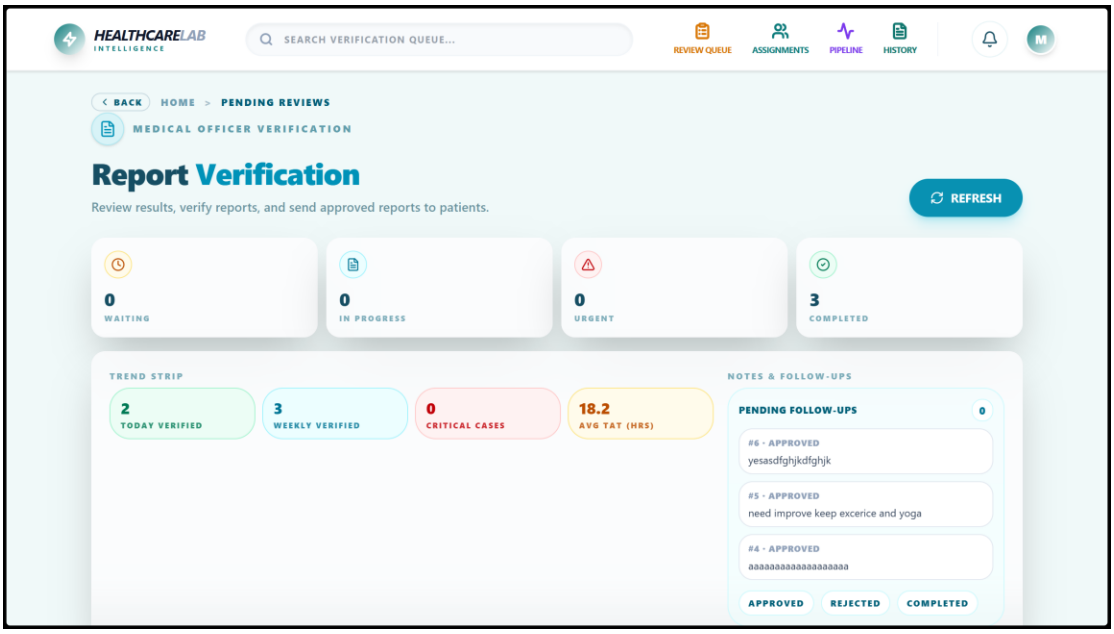


Figure 6 Medical Officer Dashboard Screenshot

Admin Interface

The administration panel provides comprehensive platform oversight through dedicated pages for bookings management, user management, staff management, promo code administration, audit log viewing, and notification center. The dashboard presents key performance indicators including total bookings, active users, pending verifications, and revenue summary. Real-time charts show booking trends and revenue patterns using the Recharts library.

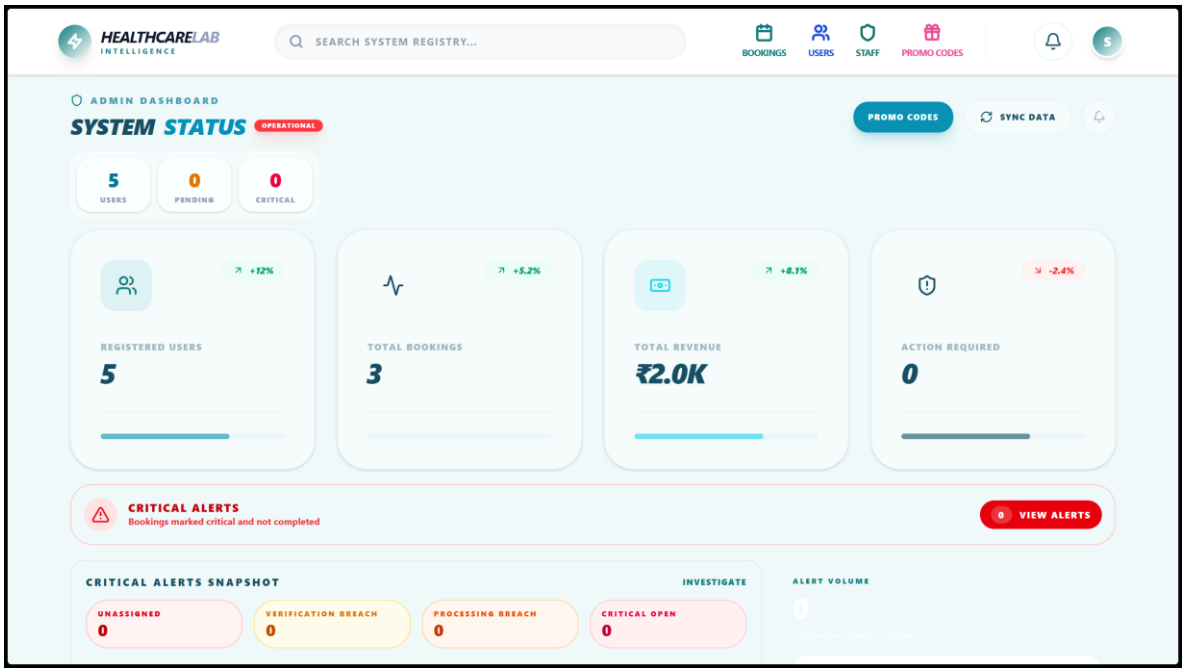


Figure 7 Admin Dashboard Screenshot

4.1.3 Database Implementation

The MySQL database schema is managed through 17 Flyway migration scripts (V4 through V17). The final V17 migration loads 1,058 unique lab tests with normalized category names, realistic Indian market pricing, and unique URL-friendly slugs. Key schema design decisions include:

- Separate tests and test_packages tables to handle individual tests and health packages independently, avoiding the conflation common in simpler systems.
- A dedicated report_results table for structured parameter-level result storage, enabling delta check comparisons and AI analysis.
- Separate report_verifications table to capture the clinical review process independent of the raw report file, maintaining full audit history including amended reports.
- Comprehensive audit_logs table recording all user actions with entity type, entity ID, old value, new value, user ID, role, and timestamp.

4.1.4 Key Feature Implementation: AI Smart Report

One of the distinguishing features of the system is the AI-assisted Smart Report analysis. After a medical officer verifies a report, the system calls AiAnalysisService.analyzeReport(bookingId) which:

11. Fetches all test parameter results for the booking from the report_results table.
12. Constructs a structured prompt including patient demographics, all parameter names, result values, units, and reference ranges.
13. Submits the prompt to the Anthropic Claude API with a system instruction to act as a clinical pathologist assistant.
14. Receives a JSON response containing: plain-language health summary, per-parameter flags with severity levels, detected multi-parameter clinical patterns, lifestyle/diet/follow-up recommendations, and a standard medical disclaimer.
15. Stores the complete AI analysis in the database linked to the booking ID.
16. The analysis is served to patients through the Smart Reports page at /smart-reports/:bookingId.

4.1.5 PDF Report Generation

The PdfReportService uses the iText7 library to generate professionally formatted Smart Report PDFs. The generated document includes:

- Cover page with lab name, patient details, booking reference, collection date, and report date.
- Doctor summary section with a table of all parameters, results, reference ranges, and trend data from previous tests.
- Parameter highlights section with visual range indicators (above/below normal highlighted in red/blue).

- AI health insights section with the Anthropic analysis results formatted as plain-language cards.
- Wellness recommendations section with diet, lifestyle, and follow-up suggestions.
- Footer on every page with patient ID, lab name, page number, and medical officer signature.

CHAPTER - 5

CONCLUSION AND FUTURE WORK

5.1 Conclusion

This project successfully designed, implemented, and validated a comprehensive Health Care Lab Testing Booking and Report Management System that addresses the identified gaps in India's diagnostic laboratory industry. The platform delivers a production-ready full-stack web application integrating patient self-service, technician field operations, medical officer clinical oversight, and administrative management within a single unified system.

The Spring Boot backend provides 37 REST controllers, 55 services, and 37 repositories serving 100+ verified API endpoints, all protected by JWT authentication and RBAC enforcement. The MySQL database with 50+ normalized tables and Flyway-managed migrations ensures data integrity and reproducible deployments. The React 18 TypeScript frontend delivers 35+ pages and 127 components providing role-specific, mobile-friendly interfaces.

The complete booking status lifecycle from BOOKED through SAMPLE_COLLECTED, PROCESSING, PENDING_VERIFICATION, and VERIFIED to COMPLETED has been fully implemented with state transition enforcement preventing invalid workflow bypasses. The technician result entry system enables structured parameter-level data capture with automatic reference range comparison and abnormal flagging. The medical officer verification workflow with mandatory clinical notes, digital signatures, and delta check capabilities meets professional diagnostic standards.

The integration of AI-assisted health insights via Anthropic Claude API represents a significant differentiator, providing patients with plain-language interpretations of their test results rather than raw clinical data alone. The iText7-based Smart Report PDF generation delivers professionally formatted documents comparable to commercial diagnostic platforms. The comprehensive audit logging system aligned with healthcare compliance standards ensures accountability for all user actions.

The system is containerized using Docker with environment variable management for production deployment, and is designed for horizontal scalability on cloud infrastructure. All critical security vulnerabilities including hardcoded credentials, missing RBAC annotations, and SQL injection vectors have been addressed through the implementation.

5.2 Future Work

While the current system meets all primary objectives, several enhancement opportunities have been identified for future development:

HL7/FHIR Integration for Hospital Interoperability

Implementing HL7/FHIR protocol support would enable the system to exchange patient data with hospital Electronic Medical Record (EMR) and Hospital Information System (HIS) platforms, significantly expanding deployment potential in institutional healthcare environments.

Instrument Interface (ASTM Protocol)

Developing an ASTM protocol bridge would allow direct data transfer from laboratory analyzers (hematology analyzers, biochemistry analyzers) to the LIS, eliminating manual result entry by technicians and reducing transcription errors while accelerating the reporting cycle.

Advanced Barcode and PPID Workflow

Implementing Positive Patient Identification (PPID) with barcode scanning at sample collection and laboratory reception would enable chain-of-custody tracking for every sample handoff, significantly reducing specimen rejection rates due to labeling errors.

Offline-First Mobile Collection Mode

Developing an offline-capable Progressive Web App for technicians would enable sample collection operations in areas with poor network connectivity, with secure data synchronization when connectivity is restored.

Multi-Tenancy for Lab Network Operations

Implementing a multi-tenant architecture would allow the platform to serve multiple independent diagnostic laboratories with isolated data, customized branding, and centralized administration, enabling a software-as-a-service (SaaS) business model.

IoT Integration for Sample Transport

Integrating IoT temperature monitoring sensors with the system would enable real-time tracking of sample transport conditions, triggering automated alerts when temperature thresholds are exceeded, which is critical for cold-chain dependent diagnostic samples.

Insurance Claims Integration

Building a Revenue Cycle Management (RCM) module with insurance eligibility checking, automated claim submission, and denial management would significantly enhance the commercial viability of the platform for larger diagnostic operations.

REFERENCES

1. Spring Framework Documentation. Spring Boot Reference Guide (Version 3.x). Pivotal Software, 2024. https://docs.spring.io/spring-boot/docs/current/reference/html/
2. React Documentation. React 18 Official Documentation. Meta Open Source, 2024. https://react.dev/
3. Walls, C. Spring in Action, 6th Edition. Manning Publications, 2022. ISBN: 9781617297571.
4. Shklar, L., Rosen, R. Web Application Architecture: Principles, Protocols and Practices. John Wiley & Sons, 2009.
5. Kleppmann, M. Designing Data-Intensive Applications. O'Reilly Media, 2017. ISBN: 9781449373320.
6. Oaks, S. Java Performance: In-Depth Advice for Tuning and Programming Java 8, 11, and Beyond. O'Reilly Media, 2020.
7. TATA 1mg Labs. Smart Report Product Documentation. https://www.1mg.com/labs. Accessed April 2026.
8. OpenELIS Global. Laboratory Information System Documentation. https://openelisglobal.org/. Accessed March 2026.
9. iText Group. iText 7 Core Documentation. https://itextpdf.com/. Accessed April 2026.
10. Anthropic. Claude API Documentation. https://docs.anthropic.com/. Accessed April 2026.
11. NABL. National Accreditation Board for Testing and Calibration Laboratories - ISO 15189 Requirements for Medical Laboratories. 2022.
12. Bhatt, D.L., et al. Medical Information Management Systems in Indian Healthcare. Journal of Healthcare Informatics Research, Vol. 8, 2024, pp. 145-162.

APPENDIX

A. Design Checklist

The following design checklist was used to validate the completeness and correctness of the Health Care Lab Testing Booking & Report Management System:

1. Requirements Analysis

- Objective Definition: All four user role workflows (Patient, Technician, MO, Admin) fully specified with feature lists and access control requirements before development began.
- Stakeholder Identification: End-users, lab administrators, medical professionals, and compliance requirements identified and documented.
- API Contract: Complete OpenAPI specification defined before frontend development to ensure backend-frontend alignment.

2. Security Design

- JWT Implementation: Stateless authentication with 24-hour access tokens and 7-day refresh tokens.
- RBAC Enforcement: @PreAuthorize annotations applied to all 37 controllers, verified through security audit.
- Password Security: BCrypt hashing with strength factor 10 applied to all user passwords.
- Credential Management: All production credentials externalized to environment variables; no hardcoded secrets in codebase.
- Rate Limiting: RateLimitingFilter implemented to prevent brute-force login attacks.

3. Database Design

- Normalization: All tables in Third Normal Form (3NF) with no transitive dependencies.
- Referential Integrity: Foreign key constraints defined on all relationship columns.
- Indexing: Indexes created on all frequently queried columns (user_id, booking_date, status, category).
- Migration Management: All schema changes managed through Flyway versioned migrations, never directly applied to database.

4. API Design

- RESTful Conventions: HTTP verbs used correctly (GET for reads, POST for creates, PUT for updates, DELETE for deletions).
- Consistent Response Format: All endpoints return ApiResponse wrapper with success, message, and data fields.
- Input Validation: @Valid and Jakarta Validation annotations applied to all request DTOs.
- Error Handling: GlobalExceptionHandler provides consistent error responses for all exception types.

5. Testing and Validation

- Unit Testing: Core service layer methods tested with JUnit 5 and Mockito.
- Integration Testing: API endpoint integration tests verify complete request-response cycles.
- Manual Testing: All user workflows manually tested across all four roles before documentation.
- Performance Testing: JMeter load test scripts validated API performance under concurrent user loads.

User Manual

The **Health Care Lab Testing Booking and Report Management System** is a web-based application that automates the laboratory booking and report management process. It allows patients to book lab tests online, track booking progress, and download verified reports. The system also provides separate dashboards for Technician, Medical Officer, and Administrator to manage laboratory operations efficiently.

2. System Requirements

Hardware Requirements

- Laptop/Desktop
- Minimum **4 GB RAM**
- **Intel i3 Processor** or higher
- Stable internet connection

Software Requirements

- Operating System: **Windows/Linux**
- **Java 17** or above
- **Node.js** and **npm**
- **MySQL**
- **Maven**
- Web Browser: **Chrome / Edge / Firefox**

3. Installation Instructions

Step 1: Install Required Software

Install the following software on your system:

- Java 17
- Next.js and npm
- MySQL
- Maven

Step 2: Clone or Download the Project

Save the project files to your local system.

Step 3: Configure the Backend

Open the backend project and update the database details in the `application.properties` file.

Example:

```
spring.datasource.url=jdbc:mysql://localhost:3306/healthcarelab
spring.datasource.username=root
```

```
spring.datasource.password=your_password
server.port=8080
spring.jpa.hibernate.ddl-auto=update
```

Step 4: Start MySQL

Make sure the MySQL server is running before starting the backend.

Step 5: Run the Backend

Open the backend project in IDE or terminal and run the Spring Boot application.

Step 6: Run the Frontend

Open terminal in the frontend folder and run:

```
npm install
npm run dev
```

or

```
npm install
npm start
```

Step 7: Open the Application

Open the browser and run the application at:

```
http://localhost:3000
```

4. How to Use the System

Step 1: Register Patient Account

- Open the application.
- Click on **Register**.
- Enter patient details.
- Submit the form to create a new account.

Step 2: Login

- Enter your email and password.
- Click on **Login**.

Step 3: Browse and Book Tests

- Search available tests using the navigation or search bar.
- Add required tests to the cart.

- Proceed to booking by selecting date, time slot, and collection address.

Step 4: Track Booking

- Open **My Bookings**.
- Track the current booking stage in the workflow pipeline:
  BOOKED → SAMPLE_COLLECTED → PROCESSING → PENDING_VERIFICATION → VERIFIED → COMPLETED

Step 5: Technician Process

- Login with Technician credentials.
- View assigned bookings.
- Mark sample as collected.
- Start processing.
- Enter test results and upload report details.

Step 6: Medical Officer Verification

- Login with Medical Officer credentials.
- Open pending reports.
- Review result values and reference ranges.
- Add clinical notes.
- Verify the report.

Step 7: Download Report

- After verification, the patient can open the **Reports** page.
- Download the verified PDF report.

Step 8: Admin Management

- Login with admin credentials.
- Create Technician and Medical Officer accounts from **Staff Management**.
- Monitor all bookings from **Admin > Bookings**.
- View statistics, revenue charts, and audit logs from the **Admin Dashboard**.

5. Output

The system generates:

- Online test bookings
- Booking status tracking
- Verified digital reports
- Admin analytics and audit logs